

38. Poser tes tuiles à la souris

Voir les carrés, tracer une zone — et le code s'écrit tout seul.

1. Ce qu'il faut comprendre

Jusqu'ici il fallait écrire `poser(6, 12, MUR);` à la main, case par case. Pour un sol de vingt cases, c'est vingt lignes, et une faute de frappe ne se voit pas.

Le **plan** ci-dessous montre les tuiles du programme telles qu'elles sont — des carrés, pas des numéros. On en choisit une, on **trace une zone** sur l'écran, et le programme reçoit ses `poser()`. La gomme remet du vide.

Le code écrit porte le **nom** de la tuile, jamais son numéro. C'est ce qui compte : un numéro devient faux le jour où l'on ajoute un dessin avant lui, alors qu'un nom reste juste. C'est le compilateur qui traduit, et c'est son travail.

L'outil ne possède que le bloc entre `// PLAN` et `// FIN DU PLAN`. Tout ce que tu écris ailleurs est à toi, et il n'y touche pas.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <Tuile>` → [0.107.2. Tuile — dessiner une case](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **476 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	49 o	← la console, à l'allumage — la mise en route de la console
main()	55 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte — attend de pouvoir écrire une case
EcrireTexte	10 o	← texte() — écrit un texte, lettre par lettre
CopierTuiles	94 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
Tuiles	176 o	← CopierTuiles — 9 lettres sur 43, et 2 tuiles dessinées
"TRACE UNE ZONE"	14 o	← texte() — un texte gravé

Lettres gravées : ACENORTUZ

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

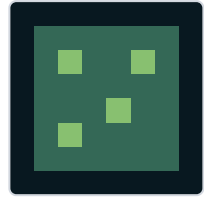
poser()	× 6	36 o sur place, dans main()
texte()	× 1	11 o sur place, dans main() + EcrireTexte (10 o)
image()	× 1	3 o sur place, dans main() + AttendreImage (28 o)

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <Tuile>    // un dessin de 8 × 8 pixels
#include <texte>     // écrit un texte à l'écran
#include <poser>     // pose une tuile sur une case du fond
```

2. UN DESSIN — MUR

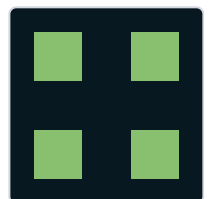
```
Tuile MUR = {
  "33333333",
  "32222223",
  "32122123",
  "32222223",
  "32221223",
  "32122223",
  "32222223",
  "33333333",
};
```



à l'écran

3. UN DESSIN — CAISSE

```
Tuile CAISSE = {
  "33333333",
  "31133113",
  "31133113",
  "33333333",
  "33333333",
  "31133113",
  "31133113",
  "33333333",
};
```



à l'écran

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
  texte(4, 1, "TRACE UNE ZONE");

  // PLAN
  poser(0, 16, MUR);
  poser(1, 16, MUR);
  poser(2, 16, MUR);
  poser(3, 16, MUR);
  poser(9, 12, CAISSE);
  poser(10, 12, CAISSE);
  // FIN DU PLAN

  while (true) {
    image();
  }

  return 0;
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Un bout de sol et deux caisses — puis ce que tu traces toi-même.

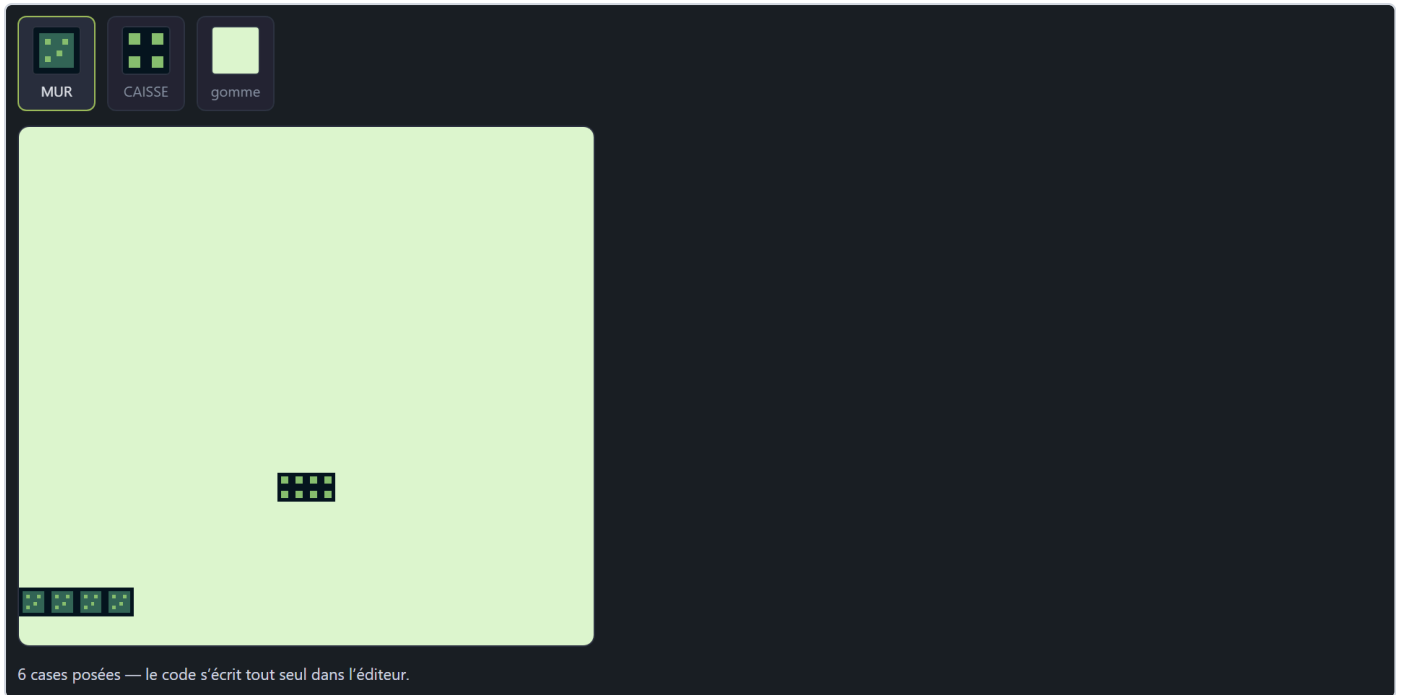
5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le sol posé est à l'écran (tuiles 44, 44, 44, 44)
- ✓ les caisses aussi, et ce n'est pas la même tuile
- ✓ rien n'est posé ailleurs
- ✓ le titre est écrit

6. La même chose, à la souris

Cette leçon porte son atelier : le geste réécrit le programme ci-dessus, et la cartouche se refait dans la seconde. C'est le même texte, écrit autrement.



Ensuite : **39. Poser une tuile** — chapitre 6 : Dessiner. · [gameboy3](#) — écrire une cartouche Game Boy en C++